

Week 7 – High-Performance Pandas and Apache Arrow

02613 Python and High-Performance Computing

Last week on 02613...

- Why should reduction-operators be commutative?
 - Why no matrix multiplication?
- The “simple” and binary-tree algorithms from last week do not reorder operands
- Many frameworks *require* this – allows for optimizations in implementation
- Some frameworks *support* non-commutative operators
 - May negatively impact performance – use only when needed

So far: array data

N

x =

4	9	1	2	3	7	1	8	5	2	0	4	6	7	1	3	2	9	2	5
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Today: tabular data

coordsx	coordsy	observed	parameterId	stationId	value
9,7875	56,1496	2023-01-17T23:59:00Z	precip_past1min	5185	0
8,1465	56,5032	2023-01-17T23:59:00Z	precip_past1min	5296	0
9,2877	57,0143	2023-01-17T23:59:00Z	precip_past1min	5085	0
8,6652	54,9872	2023-01-17T23:59:00Z	precip_past1min	5355	0
12,0981	55,2076	2023-01-17T23:59:00Z	precip_past1min	5889	0
14,7494	55,0677	2023-01-01T00:00:00Z	wind_dir_past1h	6190	209
-51,7308	64,1833	2023-01-01T00:00:00Z	temp_min_past1h	4250	-9,2
10,1272	56,3027	2023-01-01T00:00:00Z	precip_past10min	6072	0,5
9,4041	56,879	2023-01-01T00:00:00Z	precip_dur_past10min	5081	10
11,6035	55,7358	2023-01-01T00:00:00Z	precip_dur_past1h	6156	21

Today: tabular data

coordsx	coordsy	observed	parameterId	stationId	value
9,7875	56,1496	2023-01-17T23:59:00Z	precip_past1min	5185	0
8,1465	5			5296	0
9,2877	5			5085	0
8,6652	5			5355	0
12,0981	5			5889	0
14,7494	5			6190	209
-51,7308	6			4250	-9,2
10,1272	5			6072	0,5
9,4041	56,879	2023-01-01T00:00:00Z	precip_dur_past10min	5081	10
11,6035	55,7358	2023-01-01T00:00:00Z	precip_dur_past1h	6156	21



Pandas

```
>>> import pandas as pd
>>> df = pd.read_csv('data.csv')
>>> df
```

	coordsx	coordsy	observed	parameterId	stationId	value
0	9.7875	56.1496	2023-01-17T23:59:00Z	precip_past1min	5185	0.0
1	8.1465	56.5032	2023-01-17T23:59:00Z	precip_past1min	5296	0.0
2	9.2877	57.0143	2023-01-17T23:59:00Z	precip_past1min	5085	0.0
3	8.6652	54.9872	2023-01-17T23:59:00Z	precip_past1min	5355	0.0
4	12.0981	55.2076	2023-01-17T23:59:00Z	precip_past1min	5889	0.0
5	14.7494	55.0677	2023-01-01T00:00:00Z	wind_dir_past1h	6190	209.0
6	-51.7308	64.1833	2023-01-01T00:00:00Z	temp_min_past1h	4250	-9.2
7	10.1272	56.3027	2023-01-01T00:00:00Z	precip_past10min	6072	0.5
8	9.4041	56.8790	2023-01-01T00:00:00Z	precip_dur_past10min	5081	10.0
9	11.6035	55.7358	2023-01-01T00:00:00Z	precip_dur_past1h	6156	21.0

"Typical" pandas workflow

Bottleneck

1. Load data

Load speed

2. Explore data

Mem. use + speed

3. Settle on workflow

Mem. use + speed

Today

- Loading data
- Minimizing memory use
- Fast operations

About pandas

DataFrame

Row

Column

About pandas

DataFrame

	coordsx	coordsy	observed	value	← Column labels
					Row

Column

About pandas

DataFrame

	coordsx	coordsy	observed	value	← Column labels
0					
1					
2					Row
3					
4					
5					

↑
Row labels / index

Column

About pandas

```
>>> import pandas as pd
>>> df = pd.read_csv('data.csv')
>>> df['coordsx']
0      9.7875
1      8.1465
2      9.2877
3      8.6652
4     12.0981
5     14.7494
...
Name: coordsx, dtype: float64
```

Each column has its own data type

Series

0	
1	
2	
3	
4	
5	

Row labels / index

Data
(NumPy array)

About pandas

```
>>> import pandas as pd
>>> a = pd.Series([1,2,3,4])
>>> b = pd.Series([1,2,3,4])
>>> a + b
0      2
1      4
2      6
3      8
dtype: int64
>>> import numpy as np
>>> np.mean(a)
2.5
```

a

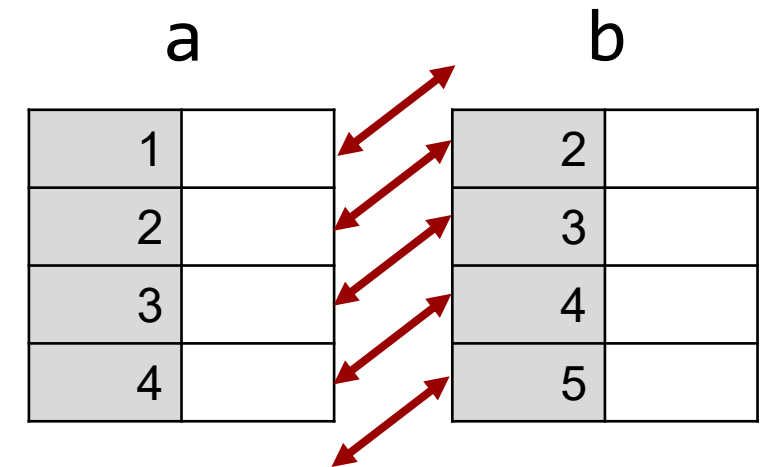
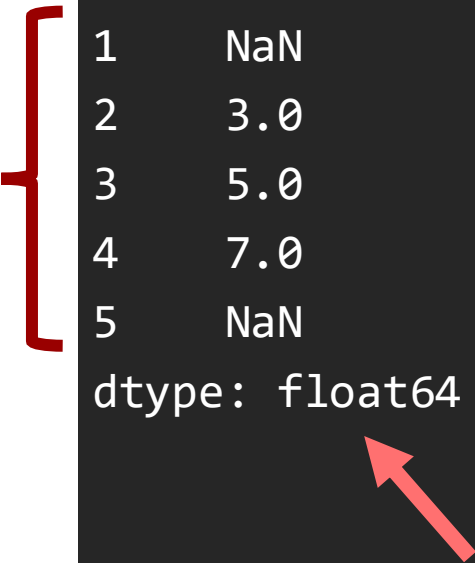
0	
1	
2	
3	

b

0	
1	
2	
3	

About pandas

```
>>> import pandas as pd
>>> a = pd.Series([1,2,3,4], index=[1,2,3,4])
>>> b = pd.Series([1,2,3,4], index=[2,3,4,5])
>>> a + b
1      NaN
2      3.0
3      5.0
4      7.0
5      NaN
dtype: float64
```



bliver float64 når vi har NaNs tror jeg

About pandas

```
>>> import pandas as pd
>>> a = pd.Series([1,2,3,4], index=[1,2,3,4])
>>> b = pd.Series([1,2,3,4], index=[2,3,4,5])
>>> a.to_numpy() + b.to_numpy() # Views
array([2, 4, 6, 8])
```

a

1	
2	
3	
4	

b

2	
3	
4	
5	

About pandas

```
>>> import pandas as pd
>>> a = pd.Series([1,2,3,4], index=[1,2,3,4])
>>> b = pd.Series([1,2,3,4], index=[2,3,4,5])
>>> df = pd.DataFrame({ 'a': a, 'b': b })
>>> df
```

	a	b
1	1.0	NaN
2	2.0	1.0
3	3.0	2.0
4	4.0	3.0
5	NaN	4.0

df

	a	b
1		
2		
3		
4		
5		

Tekst

About pandas

```
>>> import pandas as pd
>>> a = pd.Series([1,2,3,4], index=[1,2,3,4])
>>> b = pd.Series([1,2,3,4], index=[2,3,4,5])
>>> df = pd.DataFrame({ 'a': a, 'b': b })
>>> df * 2
      a      b
1  2.0  NaN
2  4.0  2.0
3  6.0  4.0
4  8.0  6.0
5  NaN  8.0
```

df

	a	b
1		
2		
3		
4		
5		

About pandas

- DataFrames are 2d tables that hold the data
- Each column is a Series with its own data type (often backed by NumPy array)
- DataFrames and Series behave like NumPy arrays
- ... except for the index!

Quiz time!

Loading data

```
$ ls
bighuge.csv.zip
$ unzip bighuge.csv.zip
$ ls
bighuge.csv.zip      bighuge.csv
$ python
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv')
```

Loading data

```
$ ls  
bighuge.csv.zip  
$ python  
>>> import pandas as pd  
>>> df = pd.read_csv('bighuge.csv.zip')
```

Faster (potentially)

Uses less disk space

Loading data

```
$ ls  
bighuge.csv.zip  
$ python  
>>> import pandas as pd  
>>> df
```

More to come!

Faster (potentially)

Uses less disk space

Memory use

```
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv.zip')
>>> df.info(memory_usage='deep')
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   coordsx         10 non-null    float64
1   coordsy         10 non-null    float64
2   observed        10 non-null    object
4   parameterId     10 non-null    object
5   stationId       10 non-null    int64
6   value           10 non-null    float64
dtypes: float64(3), int64(1), object(3)
memory usage: 2.0 GB
```

Memory use

```
>>> def summarize_columns(df):  
...     print(pd.DataFrame(  
...         (  
...             Column name  c,  
...             Data type    df[c].dtype,  
...             Num. uniq. vals  len(df[c].unique()),  
...             Mem. use [MB]  df[c].memory_usage(deep=True) // (1024**2)  
...         ) for c in df.columns  
...     ], columns=['name', 'dtype', 'unique', 'size (MB)'])  
...     print('Total size:', df.memory_usage(deep=True).sum() / 1024**2, 'MB')  
...  
>>>
```

Total mem. use of dataframe

Memory use

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	object	2134342	464
2	tpep_dropoff_datetime	object	2137286	464
3	passenger_count	float64	11	48
4	trip_distance	float64	5606	48
5	store_and_fwd_flag	object	3	352
6	DOLocationID	int64	262	48
7	payment_type	float64	6	48
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

```
Total size: 2014.113718032837 MB
```

Memory use

1. Date types  Date ending with E not A

2. Recoding categorial variables

3. Smaller numeric data types

Date types

```
>>> df['tpep_pickup_datetime']  
0          2020-01-01 00:28:15  
1          2020-01-01 00:35:39  
2          2020-01-01 00:47:41  
3          2020-01-01 00:55:23  
4          2020-01-01 00:01:58  
...  
6405003    2020-01-31 22:51:00  
6405004    2020-01-31 22:10:00  
6405005    2020-01-31 22:50:07  
6405006    2020-01-31 22:25:53  
6405007    2020-01-31 22:44:00  
Name: tpep_pickup_datetime, Length: 6405008, dtype: object
```

Issue: datetimes are stored as strings

Date types

```
>>> df["tpep_pickup_datetime"] = pd.to_datetime(df["tpep_pickup_datetime"])
>>> df["tpep_dropoff_datetime"] = pd.to_datetime(df["tpep_dropoff_datetime"])
>>> df['tpep_pickup_datetime']
0          2020-01-01 00:28:15
1          2020-01-01 00:35:39
2          2020-01-01 00:47:41
3          2020-01-01 00:55:23
4          2020-01-01 00:01:58
...
6405003    2020-01-31 22:51:00
6405004    2020-01-31 22:10:00
6405005    2020-01-31 22:50:07
6405006    2020-01-31 22:25:53
6405007    2020-01-31 22:44:00
Name: tpep_pickup_datetime, Length: 6405008, dtype: datetime64[ns]
```



Date types

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	passenger_count	float64	11	48
4	tpep_pickup_datetime	object	2134342	464
5	tpep_dropoff_datetime	object	2137286	464
6	passenger_count	float64	11	48
7	payment_type	float64	6	48
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

Total size: 1183.38600569000244 MB

Recoding categorical variables

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	passenger_count	float64	11	48
4	trip distance	float64	5606	48
5	store_and_fwd_flag	object	3	352
6	DOLocationID	int64	262	48
7	payment_type	float64	6	48
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

Total size: 1183.38600569000244 MB

Recoding categorical variables

```
>>> df['store_and_fwd_flag']
0          N
1          N
2          N
3          N
...
6405004    NaN
6405005    NaN
6405006    NaN
6405007    NaN
Name: store_and_fwd_flag, Length: 6405008, dtype: object
>>> df['store_and_fwd_flag'].unique()
array(['N', 'Y', nan], dtype=object)
```

Issues:

1. Missing values
2. True/False encoded as strings

Recoding categorical variables

```
>>> book_sol = df['store_and_fwd_flag'].fillna(" ").apply(ord)\  
... .apply(lambda x: [32, 78, 89].index(x) - 1).astype(np.int8)  
>>>
```


Recoding categorical variables

```
>>> book_sol = df['store_and_fwd_flag'].fillna(" ")
>>> book_sol.unique()
array(['N', 'Y', ' '], dtype=object)
```

Recoding categorical variables

```
>>> book_sol = df['store_and_fwd_flag'].fillna(" ").apply(ord)
>>> book_sol.unique()
array([78, 89, 32])
>>> help(ord)
Help on built-in function ord in module builtins:

ord(c, /)
    Return the Unicode code point for a one-character string.

>>> ord('a'), ord('b')
(97, 98)
```

Recoding categorical variables

```
>>> book_sol = df['store_and_fwd_flag'].fillna(" ").apply(ord)\  
... .apply(lambda x: [32, 78, 89].index(x) - 1)  
>>> book_sol.unique()  
array([ 0,  1, -1])
```

Recoding categorical variables

```
>>> book_sol = df['store_and_fwd_flag'].fillna(" ").apply(ord)\  
... .apply(lambda x: [32, 78, 89].index(x) - 1).astype(np.int8)  
>>> book_sol.unique()  
array([ 0,  1, -1], dtype=int8)
```

Recoding categorical variables

```
>>> book_sol = df['store_and_fwd_flag'].fillna(" ")\  
... .apply(lambda x: [32, 78, 89].index(x) - 1).astype(np.int8)  
>>> book_sol.unique()  
array([ 0,  1, -1], dtype=int8)  
>>>  
>>> book_sol.memory_usage(deep=True) / 1024**2 # MB  
6.108417510986328  
>>>  
>>> df['store_and_fwd_flag'].memory_usage(deep=True) / 1024**2 # MB  
352.6583957672119
```

Recoding categorical variables

```
>>> pandas_sol = df['store_and_fwd_flag'].astype('category')
>>> pandas_sol.unique()
['N', 'Y', NaN]
Categories (2, object): ['N', 'Y']
>>> pandas_sol.memory_usage(deep=True) / 1024**2 # MB
6.108631134033203
>>>
>>> book_sol.memory_usage(deep=True) / 1024**2 # MB
6.108417510986328
>>>
>>> pandas_sol.cat.codes.unique()
array([ 0,  1, -1], dtype=int8)
```

Recoding categorical variables

- Categorical variables = can take one of a fixed limited set of values
- Recoding = assign each value to a (possibly arbitrary) number, then replace values with numbers
- Can be done manually or with pandas 'category' type

Recoding categorical variables

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	passenger_count	float64	11	48
4	trip_distance	float64	5606	48
5	store_and_fwd_flag	category	3	6
6	DOLocationID	int64	262	48
7	payment_type	float64	6	48
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48
Total size: 836.8362922668457 MB				



Numeric data types



```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	passenger_count	float64	11	48
4	trip_distance	float64	5606	48
5	store_and_fwd_flag	category	3	6
6	DOLocationID	int64	262	48
7	payment_type	float64	6	48
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

Total size: 836.8362922668457 MB

Numeric data types

```
>>> df["payment_type"].unique()
array([ 1.,  2.,  4.,  3.,  5., nan])
>>> df["payment_type"] = df["payment_type"].fillna(0.0).astype(np.int8)
```

Numeric data types

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	passenger_count	float64	11	48
4	trip_distance	float64	5606	48
5	store_and_fwd_flag	category	3	6
6	DOLocationID	int64	262	48
7	payment_type	int8	6	6
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

Total size: 794.0782508850098 MB



Numeric data types

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	store_and_fwd_flag	float64	11	48
4	store_and_fwd_flag	float64	11	48
5	store_and_fwd_flag	float64	11	48
6	DOLocationID	int64	20	48
7	payment_type	int8	6	6
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

Total size: 794.0782508850098 MB

When does this work?



Numeric data types

Integers (whole numbers)

int8

int16

int32

int64

Unsigned integers (non-negative)

uint8

uint16

uint32

uint64

Floating point (fractional numbers)

float16

float32

float64

Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767

```
>>> df["payment_type"].unique()  
array([ 1.,  2.,  4.,  3.,  5., nan])  
>>> df["payment_type"] = df["payment_type"].fillna(0.0).astype(np.int8)
```

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 \cdot 10^{19}$

np.iinfo

Key point: can convert to smaller integer when:

1. Values are whole numbers
2. Min/max fits the data type

Bill Gates / Net worth

127.7 billion USD

2024

Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	$-9.22 * 10^{18}$	$9.22 * 10^{18}$

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 * 10^{19}$

`np.iinfo`

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

`np.finfo`

Floating point resolution

123.4

Exponent

$1.234 \cdot 10^2$

Significant / mantissa (only has one digit left of the point)

Resolution $\approx$ nr. of digits in significant

Floating point resolution

123.4

Exponent

**Note: on the computer everything
is in base 2!**

Resolution $\approx$ nr. of digits in significant

Floating point resolution

Example:

4 digits for significant
Exponent from -4 to 4

123.4

$1.234 \cdot 10^2$

Exponent

Significant / mantissa

Resolution $\approx$ nr. of digits in significant

Floating point resolution

Example:

4 digits for significant
Exponent from -4 to 4

12340.0

$1.234 \cdot 10^4$

Exponent

Significant / mantissa

Resolution $\approx$ nr. of digits in significant

Floating point resolution

Example:

4 digits for significant
Exponent from -4 to 4

0.0001234

$1.234 \cdot 10^{-4}$

Exponent

Significant / mantissa

Resolution $\approx$ nr. of digits in significant

Floating point resolution

Example:

4 digits for significant
Exponent from -4 to 4

$$1.0 + 0.0001234$$

$$?.??? \cdot 10^?$$

Exponent

Significant / mantissa

Resolution $\approx$ nr. of digits in significant

Floating point resolution

Example:

4 digits for significant

Exponent from -4 to 4

$$1.0 + 0.0001234$$

$$1.000 \cdot 10^0 + 1.234 \cdot 10^{-4}$$

$$1.0001234 \cdot 10^0$$

Can only keep these



Floating point resolution

Example:

4 digits for significant
Exponent from -4 to 4

$$1.0 + 0.0001234$$

$$1.000 \cdot 10^0 + 1.234 \cdot 10^{-4}$$

$$1.000 \cdot 10^0$$



Rounding error

Resolution = 0.001 (like float16)

Floating point resolution

Example:

4 digits for sig

Exponent from

$$1.0 + 0.0001234$$

Note: resolution is relative to the larger number



Rounding error

Resolution = 0.001 (like float16)

Floating point resolution

Example:

4 digits for significant

Exponent from -4 to 4

$$10.0 + 0.001234$$

$$1.000 \cdot 10^1 + 1.234 \cdot 10^{-3}$$

$$1.0001234 \cdot 10^1$$

Can only keep these



Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	$-9.22 * 10^{18}$	$9.22 * 10^{18}$

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 * 10^{19}$

np.iinfo

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

```
>>> a = np.array(1.0, dtype='float16')
>>> b = np.array(0.1, dtype='float16')
>>> +a, +b # Unary + makes it print nice
(1.0, 0.1)
>>> a + b
1.1
```

np.finfo

Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	$-9.22 * 10^{18}$	$9.22 * 10^{18}$

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 * 10^{19}$

np.iinfo

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

```
>>> a = np.array(1.0, dtype='float16')
>>> b = np.array(0.001, dtype='float16')
>>> +a, +b # Unary + makes it print nice
(1.0, 0.001)
>>> a + b
1.001
```

np.finfo

Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	$-9.22 * 10^{18}$	$9.22 * 10^{18}$

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 * 10^{19}$

np.iinfo

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

```
>>> a = np.array(1.0, dtype='float16')
>>> b = np.array(0.0001, dtype='float16')
>>> +a, +b # Unary + makes it print nice
(1.0, 0.0001)
>>> a + b
1.0
```



Rounding error!

np.finfo

Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	$-9.22 * 10^{18}$	$9.22 * 10^{18}$

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 * 10^{19}$

np.iinfo

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

```
>>> a = np.array(1.0, dtype='float16')
>>> b = np.array(0.001, dtype='float16')
>>> +a, +b # Unary + makes it print nice
(1.0, 0.001)
>>> a + b
1.001
```

np.finfo

Numeric data types

Integers (whole numbers)

	min	max
int8	-128	127
int16	-32,768	32,767
int32	-2,147,483,648	2,147,483,647
int64	$-9.22 * 10^{18}$	$9.22 * 10^{18}$

Unsigned integers (non-negative)

uint8	0	255
uint16	0	65535
uint32	0	4,294,967,295
uint64	0	$1.84 * 10^{19}$

np.iinfo

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

```
>>> a = np.array(10.0, dtype='float16')
>>> b = np.array(0.001, dtype='float16')
>>> +a, +b # Unary + makes it print nice
(10.0, 0.001)
>>> a + b
10.0
```



Rounding error!

np.finfo

Numeric data types

```
>>> summarize_columns(df)
```

	name	dtype	unique	size (MB)
0	VendorID	float64	3	48
1	tpep_pickup_datetime	datetime64[ns]	2134342	48
2	tpep_dropoff_datetime	datetime64[ns]	2137286	48
3	passenger_count	float64	11	48
4	trip_distance	float64	5606	48
5	store_and_fwd_flag	category	3	6
6	DOLocationID	int64	262	48
7	payment_type	int8	6	6
8	fare_amount	float64	5283	48
9	mta_tax	float64	11	48
10	tip_amount	float64	3626	48
11	total_amount	float64	12488	48

Total size: 794.0782508850098 MB



Numeric data types

```
>>> df['fare_amount']  
0          6.00  
1          7.00  
2          6.00  
3          5.50  
4          3.50  
...  
6405003    17.59  
6405004    46.67  
6405005    48.85  
6405006    27.17  
6405007    54.56  
Name: fare_amount, Length: 6405008, dtype: float64
```


Numeric data types

```
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float32')).sum()
0.06289832668632866    we incur an error of 6 cents
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float32')).max()
2.4414062522737368e-05
>>>
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float16')).sum()
511.4022631836173    hvis vi bruger 16 bit float så bliver 511 dollars
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float16')).max()
1.0    på individuelle transactions er forskellen på dollar niveau
```

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

Numeric data types

```
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float32')).sum()
0.06289832668632866
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float16')).sum()
2.4414062522737368e-05
>>>
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float16')).max()
511.4022631836173
>>> np.abs(df['fare_amount'] - df['fare_amount'].astype('float16')).max()
1.0
```

Key point: must check for errors before using smaller float type.

Floating point (fractional numbers)

	resolution	min/max
float16	0.001	$\pm 6.55 * 10^4$
float32	1e-6	$\pm 3.40 * 10^{38}$
float64	1e-15	$\pm 1.80 * 10^{308}$

Memory use

1. Date types

2. Recoding categorical variables

3. Smaller numeric data types

Faster operations – row wise computations

```
def get_tip_mean_explicit(df):  
    all_tips = 0  
    for i in range(len(df)): # len(df) = num. rows  
        row = df.iloc[i]      # Access row by index  
        all_tips += row["tip_amount"] / row["total_amount"]  
  
    return all_tips / len(df)  
  
t = time()  
get_tip_mean_explicit(df)  
print(time() - t)  
# LOOOOOOOONG (minutes)
```

Slow but simple

Faster operations – row wise computations

```
def get_tip_mean_apply(df):  
    frac_tip = df.apply(  
        lambda row: row["tip_amount"] / row["total_amount"],  
        axis=1 # Apply to each row, i.e., apply along columns (axis 1)  
    )  
    return frac_tip.mean() # NumPy like syntax  
  
t = time()  
get_tip_mean_apply(df)  
print(time() - t)  
# 9.5 s
```

Fast-ish and general

Faster operations – row wise computations

```
def get_tip_mean_vectorized(df):  
    # Divide columns with NumPy-like syntax  
    frac_tip = df["tip_amount"] / df["total_amount"]  
    return frac_tip.mean()
```

```
t = time()  
get_tip_mean_vectorized(df)  
print(time() - t)  
# 32 ms
```

Fastest

Faster operations – row wise computations

```
def get_tip_mean_numpy(df):  
    tip_amount_np = df["tip_amount"].to_numpy()  
    total_amount_np = df["total_amount"].to_numpy()  
    # Pure NumPy from here on  
    frac_tip = tip_amount_np / total_amount_np  
    return frac_tip.mean()
```

```
t = time()  
get_tip_mean_numpy(df)  
print(time() - t)  
# 11 ms
```

Fastester!

Faster operations – using the index

DataFrame

	coordsx	coordsy	observed	value
0				
1				
2				
3				
4				
5				

← Column labels

↑
Row labels / index

Faster operations – using the index

DataFrame

	coordsx	coordsy	value
observed			
2023-01-17T23:59:00Z			
2023-01-17T23:59:00Z			
2023-01-17T23:59:00Z			
2023-01-17T23:59:00Z			
2023-01-17T23:59:00Z			
2023-01-01T00:00:00Z			

← Column labels

↑
Row labels / index

Faster operations – using the index

```
>>> df[ # Who paid congestion surcharge at given time \  
...     (df["tpep_pickup_datetime"] == "2020-01-06 08:13:00") & \  
...     (df["congestion_surcharge"] > 0)  
... ]  
>>>
```



Boolean indexing

Faster operations – using the index

```
>>> df[ # Who paid congestion surcharge at given time \  
...     (df["tpep_pickup_datetime"] == "2020-01-06 08:13:00") & \  
...     (df["congestion_surcharge"] > 0)  
... ]  
>>>
```

35 ms

Faster operations – using the index

```
>>> df[ # Who paid congestion surcharge at given time \
...      (df["tpep_pickup_datetime"] == "2020-01-06 08:13:00") & \
...      (df["congestion_surcharge"] > 0)
... ]
>>>
>>> df_indexed = df.set_index("tpep_pickup_datetime")
>>>
>>> df_time = df_indexed.loc["2020-01-06 08:13:00"]
>>> df_time[df_time["congestion_surcharge"] > 0]
>>>
```

35 ms

9.3 ms

Faster operations – using the index

```
>>> df[ # Who paid congestion surcharge at given time \
...     (df["tpep_pickup_datetime"] == "2020-01-06 08:13:00") & \
...     (df["congestion_surcharge"] > 0)
... ]
>>>
>>> df_indexed = df.set_index("tpep_pickup_datetime")
>>>
>>> df_time = df_indexed.loc["2020-01-06 08:13:00"]
>>> df_time[df_time["congestion_surcharge"] > 0]
>>>
>>> df_indexed_sorted = df_indexed.sort_index()
>>>
>>> df_time = df_indexed_sorted.loc["2020-01-06 08:13:00"]
>>> df_time[df_time["congestion_surcharge"] > 0]
>>>
```

35 ms

9.3 ms

597 μ s

Faster operations – using the index

```
>>> df[ # Who paid congestion surcharge at given time \
...     (df["tpep_pickup_datetime"] == "2020-01-06 08:13:00") & \
...     (df["congestion_surcharge"] > 0)
... ]
```

35 ms

Key point: indexing significantly speeds up queries.

Price: we must first build the index

```
>>> df_indexed = df.set_index("tpep_pickup_datetime")
```

```
>>>
```

```
>>> df_time = df_indexed.loc["2020-01-06 08:13:00"]
```

```
>>> df_time[df_time["congestion_surcharge"] > 0]
```

```
>>>
```

```
>>> df_indexed_sorted = df_indexed.sort_index()
```

```
>>>
```

```
>>> df_time = df_indexed_sorted.loc["2020-01-06 08:13:00"]
```

```
>>> df_time[df_time["congestion_surcharge"] > 0]
```

```
>>>
```

9.3 ms

597 µs

Quiz time!

Loading data

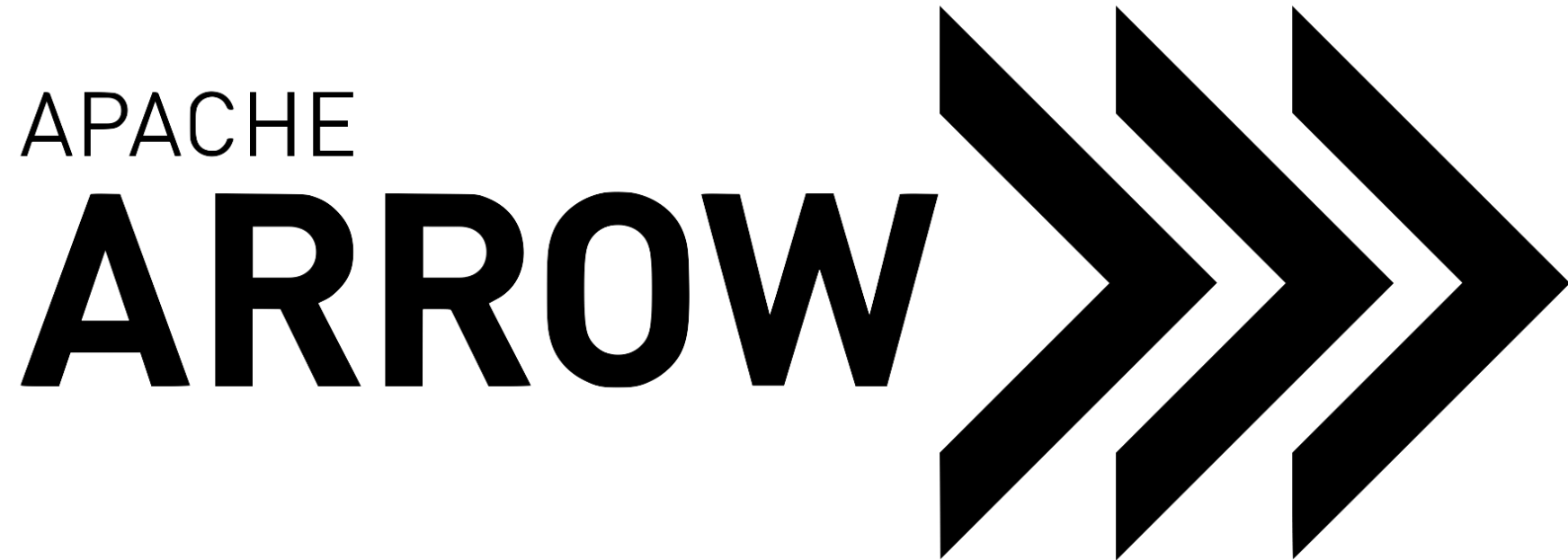
```
$ ls  
bighuge.csv.zip  
$ python  
>>> import pandas as pd  
>>> df
```

More to come!

Faster (potentially)

Uses less disk space

PyArrow for faster loading



PyArrow for faster loading

```
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv')      # 11.3 s
>>>
>>> from pyarrow import csv
>>> table = csv.read_csv('bighuge.csv')  # 4.6 s
```

PyArrow for faster loading

```
>>> df
...
0    VendorID          float64
1    tpep_pickup_datetime object
2    tpep_dropoff_datetime object
3    passenger_count      float64
4    trip_distance        float64
5    RatecodeID          float64
6    store_and_fwd_flag   object
7    PULocationID        int64
8    DOLocationID        int64
9    payment_type         float64
10   fare_amount          float64
11   extra                float64
...
```

```
>>> table
pyarrow.Table
VendorID: int64
tpep_pickup_datetime: timestamp[s]
tpep_dropoff_datetime: timestamp[s]
passenger_count: int64
trip_distance: double
RatecodeID: int64
store_and_fwd_flag: string
PULocationID: int64
DOLocationID: int64
payment_type: int64
fare_amount: double
extra: double
...
```

PyArrow for faster loading

```
>>> df
```

```
...
```

```
0 Vend
```

```
1 tpep
```

```
2 tpep
```

```
3 pass
```

```
4 trip
```

```
5 Rate
```

```
6 stor
```

```
7 PULo
```

```
8 DOLo
```

```
9 paym
```

```
10 fare
```

```
11 extr
```

```
...
```

```
>>> table
```

```
pyarrow.Table
```

```
stamp[s]
```

```
stamp[s]
```

Key points:

1. PyArrow handles more data types (e.g., dates) automatically.
2. PyArrow has specialized data structures for handling missing data

PyArrow for faster loading

```
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv')      # 11.3 s
>>>
>>> from pyarrow import csv
>>> table = csv.read_csv('bighuge.csv')  # 4.6 s
>>>
>>> df2 = table.to_pandas()
```

PyArrow for faster loading

```
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv')      # 11.3 s
>>>
>>> from pyarrow import csv
>>> table = csv.read_csv('bighuge.csv')  # 4.6 s
>>>
>>> import pyarrow as pa
>>> copts = csv.ConvertOptions(
...     column_types={
...         "VendorID": pa.bool_(),
...         "store_and_fwd_flag": pa.dictionary(pa.int32(), pa.string()),
...     }
... )
>>> table = csv.read_csv('bighuge.csv', convert_options=copts)
```

PyArrow for faster loading

```
>>> import pandas as pd
>>> df = pd.read_csv('bighuge.csv')      # 11.3 s
>>>
>>> from pyarrow import csv
>>> table = csv.read_csv('bighuge.csv')  # 4.6 s
```

See more in the book and the official documentation

```
...     "VendorID": pa.bool_(),
...     "store_and_fwd_flag": pa.dictionary(pa.int32(), pa.string()),
... }
... )
>>> table = csv.read_csv('bighuge.csv', convert_options=copts)
```

Parquet files for even faster loading

```
1, 2020-01-01 00:28:15, 2020-01-01 00:33:03, 1, 1.20, 1, N, 238, 239, 1, 6, 3, 0.5, 1.47, 0, 0.3, 11.27, 2.5
1, 2020-01-01 00:35:39, 2020-01-01 00:43:04, 1, 1.20, 1, N, 239, 238, 1, 7, 3, 0.5, 1.5, 0, 0.3, 12.3, 2.5
1, 2020-01-01 00:47:41, 2020-01-01 00:53:52, 1, .60, 1, N, 238, 238, 1, 6, 3, 0.5, 1, 0, 0.3, 10.8, 2.5
1, 2020-01-01 00:55:23, 2020-01-01 01:00:14, 1, .80, 1, N, 238, 151, 1, 5.5, 0.5, 0.5, 1.36, 0, 0.3, 8.16, 0
2, 2020-01-01 00:01:58, 2020-01-01 00:04:16, 1, .00, 1, N, 102, 102, 2, 2.5, 0.5, 0.5, 0, 0, 0.3, 4.8, 0
```

Issues:

- Text wastes space when storing data
- Text requires parsing when reading / writing

```
1, 2020-01-01 00:56:27, 2020-01-01 01:21:44, 1, 3.30, 1, N, 161, 144, 1, 17, 3, 0.5, 4.15, 0, 0.3, 24.95, 2.5
2, 2020-01-01 00:21:54, 2020-01-01 00:27:31, 1, 1.07, 1, N, 43, 239, 1, 6, 0.5, 0.5, 1.96, 0, 0.3, 11.76, 2.5
2, 2020-01-01 00:38:01, 2020-01-01 01:15:21, 1, 7.76, 1, N, 143, 25, 1, 28.5, 0.5, 0.5, 4.84, 0, 0.3, 37.14, 2.5
1, 2020-01-01 00:15:35, 2020-01-01 00:27:06, 3, 1.60, 1, N, 211, 234, 2, 9, 3, 0.5, 0, 0, 0.3, 12.8, 2.5
1, 2020-01-01 00:41:20, 2020-01-01 00:44:22, 1, .50, 1, Y, 234, 90, 1, 4, 3, 0.5, 1, 0, 0.3, 8.8, 2.5
1, 2020-01-01 00:56:38, 2020-01-01 01:13:34, 1, 1.70, 1, N, 246, 142, 2, 11.5, 3, 0.5, 0, 0, 0.3, 15.3, 2.5
2, 2020-01-01 00:08:21, 2020-01-01 00:25:29, 1, 8.45, 1, N, 138, 216, 2, 24.5, 0.5, 0.5, 0, 0, 0.3, 25.8, 0
1, 2020-01-01 00:25:39, 2020-01-01 00:27:05, 1, .00, 1, N, 170, 162, 4, 3, 3, 0.5, 0, 0, 0.3, 6.8, 2.5
```


Parquet files for even faster loading

```
>>> import pyarrow as pa
>>> from pyarrow import csv
>>> import pyarrow.parquet as pq
>>>
>>> table = csv.read_csv('bighuge.csv') # 4.6 s
>>>
>>> pq.write_table(table, 'bighuge.parquet')
>>>
>>> pq.read_table('bighuge.parquet') # 719 ms

$ ls -lh
-rw-r--r-- 1 patmjen s14 567M Dec 26 2022 bighuge.csv
-rw-r--r-- 1 patmjen s14 111M Jan 26 13:52 bighuge.csv.zip
-rw-r--r-- 1 patmjen s14 108M Mar 12 18:11 bighuge.parquet
```

Parquet files for even faster loading

```
>>> import pyarrow as pa
>>> from pyarrow import csv
>>> import pyarrow.parquet as pq
```

```
>>>
```

```
>>> table
```

```
>>>
```

```
>>> pq.wr:
```

```
>>>
```

```
>>> pq.read_table('bighuge.parquet') # 719 ms
```

```
$ ls -lh
```

```
-rw-r--r-- 1 patmjen s14 567M Dec 26 2022 bighuge.csv
```

```
-rw-r--r-- 1 patmjen s14 111M Jan 26 13:52 bighuge.csv.zip
```

```
-rw-r--r-- 1 patmjen s14 108M Mar 12 18:11 bighuge.parquet
```

Key point: Binary parquet files are much smaller and much faster

Quiz time!

Today's exercise

Pandas and PyArrow

- Reading files with pandas
- Reading files with PyArrow
- Reading files with Parquet
- Vectorized and indexed operations

3. Check the columns and list all the changes you can do to save memory. Hint: check sections 7.1.2 and 7.1.3 on Fast Python for inspiration. You can use the following function summarize the dataframe:

```
def summarize_columns(df):  
    print(pd.DataFrame([  
        (  
            c,  
            df[c].dtype,  
            len(df[c].unique()),  
            df[c].memory_usage(deep=True) // (1024**2)  
        ) for c in df.columns  
    ], columns=['name', 'dtype', 'unique', 'size (MB)']))  
    print('Total size:', df.memory_usage(deep=True).sum() / 1024**2, 'MB')
```



Useful concepts

Read with pandas

```
import pandas as pd  
df = pd.read_csv('data.csv')
```

Read with PyArrow

```
from pyarrow import csv  
table = csv.read_csv('data.csv')
```

```
df = table.to_pandas() # Convert back to pandas
```

Read with Parquet

```
import pyarrow.parquet as pq  
table = pq.read_table('data.csv')
```

Change to work node

```
linuxsh
```

Submit job script

```
bsub < submit.sh
```

Job status

```
bstat / bjobs
```

Check job output

```
bpeek / bpeek <JOBID>
```

Kill job

```
bkill <JOBID>
```